

Onboarding NG-KV to Your Inference Stack

Trace-first integration: certify the policy on your own workload offline, then bind the gate. No production risk until step 5.

NG-KV v0.2 · Apache 2.0 · `pip install numpy; unzip; go` · Syni open research stack · July 2026

Step 1 — Capture traces from your stack (pick the cheapest signal you have)

A trace is one request's per-step attention over its KV cache, at whatever granularity you can afford. Token-level is ideal; block-level (16–64 tokens) is fine and is what production paths use — coarser granularity only weakens, never invalidates, the regret bound. You do NOT need to materialize full attention (FlashAttention won't give it to you anyway). Any of these work:

Eager replay: re-run a sample of logged requests offline with eager attention (HF adapter included) — zero prod changes.

Block statistics: Quest-style per-block key min/max against the query gives a cheap per-block attention upper bound, online.

Cache telemetry: prefix/radix hit counts and block touch timestamps you likely already log approximate mass + recency.

```
from ngkv.traces import save_trace          # .npz schema: attn (D,total), prompt_len, meta
save_trace('traces/req_001.npz', attn, prompt_len=P,
           meta={'model':'glm-5.2','granularity':32,'reduction':'max(heads)'})
```

Step 2 — Replay offline: quality, regret, significance (minutes, no GPU)

One command evaluates sliding-window, accumulated-mass, NG scoring, HBM/DRAM tiering, and mixed precision at every budget, against the clairvoyant oracle. Output: attention-mass recovered, plug-in regret per family, paired Wilcoxon + bootstrap CIs.

Read it like this: low NG regret = the estimator is near-optimal, remaining loss is the policy family - change action, not scorer.

High NG regret = headroom in the scorer - tune NecessityConfig (ewma_alpha, variance/recency weights) on your traces.

```
python replay_real.py      # reads traces/*.npz -> results_real.json + console report
```

Step 3 — Pick the action the replay justifies

Sharp attention (regret gap large, oracle high): necessity-ranked eviction or tiering captures most value cheaply.

Diffuse attention (oracle itself low): eviction cannot win - use mixed precision; in our real traces it beat the eviction oracle.

Disaggregated (P/D split, EFA/RDMA): tier by necessity; EVICT-tier blocks are never shipped (60% interconnect bytes saved at B=0.2 in simulation) and prefetch runs in descending necessity so decode starts before the tail arrives.

Step 4 — Bind the gate (thin adapters, core is stateless-simple)

vLLM: implement KVConnector with NG-KV as admission (should_offload -> Tier) + prefetch ordering. Score a block as the MAX necessity of its tokens - one vital token protects its block (the conservative choice). ~200 LoC against a pinned version.

SGLang: HiCache admission + eviction ranking over radix nodes. LFU is NG with variance=recency=0, so rollback is trivial.

TensorRT-LLM / custom: the scorer is plain NumPy, O(T) state, microseconds/step - embed it wherever eviction order is decided.

MoE stacks (GLM / DeepSeek / Qwen): router entropy is a free per-step necessity signal; sinks are pinned by default.

```
from ngkv import NecessityScorer, TieredPlacementPolicy, TierBudget
scorer, policy = NecessityScorer(), TieredPlacementPolicy(TierBudget(hbm_frac=.3, dram_frac=.3))
scorer.observe(block_attn_estimate)          # any per-step signal, any granularity
placement = policy.place(scorer.scores())    # Tier.HBM / Tier.DRAM / Tier.EVICT
```

Step 5 — Ship behind gates, keep the regret loop running

A/B with SLA-goodput as primary, per-request quality as guardrail; diagnostics: effective hit rate, bytes/output token, TTFT p50/p99 on hit and miss paths. Keep replaying fresh trace samples offline - regret drift is your early warning that the workload's attention distribution moved before the goodput metric shows it.

Stated plainly

Published numbers are attention-mass proxies from small-scale traces - the pipeline exists so you can replace them with your own workload's numbers before believing anything. Validate end-task quality behind gates, not mass proxies. Fidelity factors for quantized tiers are model-dependent: measure yours. The HF adapter is for validation, not speed.